Simulação de uma missão de busca e salvamento em Marte

Alexandre Coelho e Gonçalo Noronha
Instituto Superior de Engenharia do Porto
Porto, Portugal

***Resumo*—Este trabalho apresenta um sistema multirrobótico simulado em que um drone X500 procura o módulo de pouso Pathfinder e fornece a sua posição a um rover Perseverance. O drone sobrevoa a área de busca com o autopilot PX4 e utiliza um detetor YOLO para identificar o alvo nas imagens da câmara. A deteção é projetada sobre o terreno e a posição estimada é publicada num tópico ROS2. O rover subscreve esse tópico, calcula um ponto de aproximação junto do alvo e navega até esse ponto usando a pose disponibilizada pelo Gazebo. Num ensaio integrado, os dois robôs executaram esta sequência autonomamente. A avaliação foi qualitativa e incidiu num único ponto de interesse, num percurso em que a deteção de obstáculos não foi acionada.**

***Palavras-chave*—Sistemas multirrobóticos, ROS2, Gazebo, PX4, YOLO, navegação autónoma.**

I. Problema

A. Contexto e objetivo

A missão Mars 2020 da NASA levou o rover Perseverance a Marte [1], [2]. A missão incluiu o Ingenuity, um helicóptero experimental transportado pelo rover [3], [4]. A combinação de um veículo aéreo com um veículo terrestre inspirou o cenário simulado deste trabalho. Neste cenário, o módulo de pouso da missão Mars Pathfinder foi representado como o objeto a localizar [5], [6].

O requisito central do projeto de Redes de Sensores Distribuídos era pôr dois robôs a operar e a comunicar entre si. O grupo definiu uma missão em que o robô aéreo recolhe informação sobre um alvo e o rover usa essa informação para decidir para onde se deslocar. A proposta inicial previa que o Ingenuity identificasse vários pontos de interesse, publicasse uma lista ordenada por prioridade e que o Perseverance planeasse sucessivamente a deslocação entre esses pontos. O âmbito demonstrado foi reduzido a um único ponto de interesse (POI) e a função de reconhecimento aéreo passou para o X500.

B. Requisitos da missão demonstrada

O X500 devia sobrevoar a área de busca, identificar o Pathfinder e disponibilizar a sua posição. O Perseverance devia receber essa posição e chegar autonomamente a um ponto próximo do objeto, com afastamento suficiente para evitar contacto com o modelo. A integração devia permitir executar a sequência sem introduzir manualmente as coordenadas do alvo durante o ensaio.

II. Solução

A. Cenário e componentes reutilizados

O cenário reutilizou o terreno marciano e o modelo do rover Perseverance do pacote `spaceros_gz_demos` [7]. Para representar o módulo de aterragem Mars Pathfinder, foi descarregado do 3D Warehouse do SketchUp o modelo *Mars Pathfinder Lander (HIGH Detail)* [8]. A malha, em formato COLLADA (`.dae`), foi organizada na pasta `mars_pathfinder`, juntamente com os ficheiros `model.config` e `model.sdf`. No cenário, o Pathfinder foi colocado como um alvo estático dentro da área de busca do X500. A simulação foi executada com ROS2 Jazzy e Gazebo Harmonic [9]–[12]. O mundo e os modelos foram descritos em SDFormat [13]. A ponte `ros_gz_bridge` disponibilizou ao ROS2 a pose e os dados da câmara de profundidade do rover e encaminhou para o Gazebo os comandos de velocidade [14]. Os nós de controlo foram desenvolvidos em C++ com `rclcpp` [15].

B. X500 e localização do Pathfinder

Controlo de voo. O X500 percorre a área de busca seguindo pontos de passagem. O nó de missão envia referências de posição ao PX4 em modo *Offboard*. O autopiloto trata do controlo de voo a partir dessas referências [16]. Na configuração de arranque do X500, o Micro XRCE-DDS Agent liga o cliente PX4 à rede DDS. A interface disponibiliza nos tópicos `/fmu/out/` os dados de posição local e atitude usados para localizar o alvo [17], [18].

Deteção na imagem. O YOLO processa cada imagem da câmara e estima caixas delimitadoras e pontuações de confiança para as classes detetadas [19]. Neste projeto, a classe de interesse é o Pathfinder. A deteção indica onde o objeto aparece na imagem. O passo seguinte usa esse ponto em píxeis para estimar a posição física do alvo.

Projeção para o mundo. No pacote `x500_control`, o nó `PoiLocator` recebe a deteção e usa a posição e a atitude provenientes do PX4 para transformar o raio de observação para o referencial do mundo. A estimativa corresponde à interseção desse raio com a representação da elevação do terreno. A transformação tem em conta a convenção NED (norte, este, baixo) usada pelo PX4 e a convenção ENU (este, norte, cima) usada na representação do mundo [17]. Deteções sucessivas que resultem em posições próximas permitem confirmar o alvo. O ponto confirmado é publicado em `/x500/poi` como uma mensagem `geometry_msgs/msg/PointStamped`,

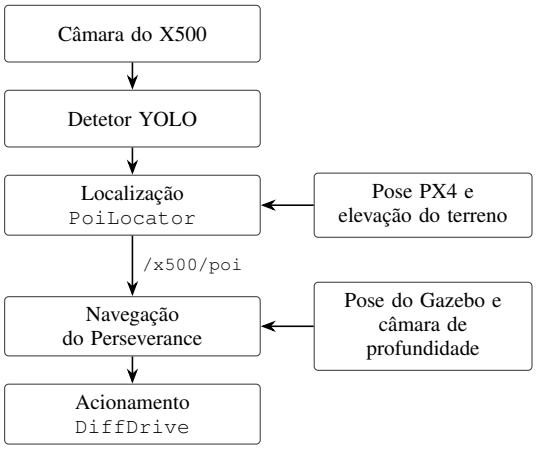


Fig. 1. Fluxo de informação desde a deteção aérea até ao acionamento do rover.

com o referencial `world`. O nó pode disponibilizar as estimativas intermédias em `/x500/blob_world`, enquanto o rover recebe o ponto confirmado no tópico de missão [20].

C. Perseverance e navegação terrestre

Posição e acionamento. O controlador terrestre utiliza a pose de referência do Gazebo (*ground truth*), publicada pelo `OdometryPublisher` e disponibilizada em ROS2 no tópico /perseverance/ground_truth/odometry como `nav_msgs/msg/Odometry` [21], [22]. O grupo reconfigurou o plugin `DiffDrive` para comandar as seis rodas. O modelo inicial acionava apenas as duas rodas dianteiras.

Aproximação ao alvo. O nó subscreve `/x500/poi` e calcula um destino nas imediações do Pathfinder, com um afastamento em relação ao ponto recebido. O controlo usa a diferença de posição no plano horizontal. Seja (x, y) a posição do rover, ψ a sua orientação em torno do eixo vertical e (x_a, y_a) o ponto de aproximação. A distância ao destino e o erro de orientação são dados por

$$d = \sqrt{(x_a - x)^2 + (y_a - y)^2}, \tag{1}$$

$$\psi_a = \operatorname{atan2}(y_a - y, x_a - x), \tag{2}$$

$$e_\psi = \operatorname{atan2}\big(\sin(\psi_a - \psi),\, \cos(\psi_a - \psi)\big). \tag{3}$$

A velocidade angular corrige o erro de orientação. O controlador autoriza o avanço quando o rover está alinhado dentro da tolerância angular e termina a aproximação quando a distância ao destino fica dentro da tolerância de chegada. O nó do pacote `perseverance_navigation` publica os comandos de velocidade linear e angular em /perseverance/cmd_vel através de mensagens `geometry_msgs/msg/Twist` [23].

Deteção de obstáculos. O nó de navegação recebe a nuvem de pontos da câmara de profundidade e verifica se há pontos suficientes numa região à frente do rover. A lógica pode ordenar a paragem perante um obstáculo nessa região. A interface de profundidade utiliza mensagens `sensor_msgs/msg/PointCloud2` [24].

Tabela I
RESULTADOS DO ENSAIO INTEGRADO

Requisito ou função	**Resultado observado**
Reconhecimento aéreo	O X500 identificou o Pathfinder durante o sobrevoo.
Comunicação entre robôs	O rover recebeu as coordenadas publicadas pelo subsistema aéreo.
Aproximação autónoma	O Perseverance chegou ao ponto de aproximação calculado a partir da mensagem.
Paragem perante obstáculos	A deteção estava ativa. O percurso ensaiado não apresentou obstáculos relevantes que a acionassem.

D. Integração entre os robôs

A Fig. 1 mostra o fluxo de informação entre a deteção aérea e o controlo terrestre. A publicação do ponto num tópico ROS 2 separa a produção da estimativa do seu uso pelo rover [25]. O referencial comum permite interpretar a posição do alvo e a pose do Perseverance no mesmo sistema de coordenadas. As coordenadas são disponibilizadas pelo tópico durante a operação, sem exportação para CSV no fluxo final.

III. RESULTADOS

A. Ensaio integrado

O ensaio integrado percorreu a sequência completa numa única execução. O X500 sobrevoou a área, identificou o Pathfinder e publicou as coordenadas estimadas. O Perseverance recebeu a mensagem, calculou o ponto de aproximação e chegou autonomamente ao destino. A Tabela I compara os requisitos com o que foi observado.

B. Discussão

Durante o desenvolvimento do Perseverance, observou-se uma divergência entre a rotação física no simulador e a estimativa da odometria das rodas. Num ensaio de diagnóstico, a variação de orientação foi de cerca de $5{,}5^\circ$ segundo a pose do Gazebo e de 51° segundo a odometria das rodas. O ensaio mostrou que a orientação estimada pelas rodas podia divergir substancialmente da orientação do modelo. Na configuração funcional, o controlador passou a usar diretamente a pose disponibilizada pelo simulador. Esta escolha facilitou a integração, mas torna a localização do rover dependente do acesso ao estado simulado.

A avaliação da missão final foi qualitativa. O ensaio demonstra a troca de informação e a execução da aproximação no cenário utilizado. Ficaram por quantificar a precisão da estimativa aérea, o erro final do rover e o tempo de execução. O trajeto não apresentou obstáculos relevantes para validar a paragem automática. A navegação em terreno montanhoso ficou por concluir. A possível influência do `model.sdf` nas dificuldades do rover ficou por investigar.

IV. Conclusão

O sistema desenvolvido demonstrou a cooperação entre um X500 e um Perseverance numa missão simulada de reconhecimento e aproximação. A estimativa aérea do ponto de interesse passou do drone para o rover através do ROS 2 e foi usada para completar a deslocação sem introdução manual do destino durante a execução.

Como trabalho futuro, propõe-se retomar a identificação de vários pontos de interesse e a sua ordenação por prioridade, validar a paragem perante obstáculos em ensaios dedicados e investigar a mobilidade do rover em terreno inclinado. Para planear percursos em terreno irregular, podem estudar-se grelhas que representem a elevação e a transitabilidade [26]. A pesquisa heurística e os métodos de planeamento global estudados para os Mars Exploration Rovers são referências para essa extensão [27], [28].

Referências

[1] K. A. Farley *et al.*, "Mars 2020 Mission Overview," *Space Science Reviews*, vol. 216, 2020, art. no. 142. [Online]. Available: https://doi.org/10.1007/s11214-020-00762-y

[2] NASA, "Mars 2020: Perseverance Rover," consultado em 8 nov. 2026. [Online]. Available: https://science.nasa.gov/mission/mars-2020-perseverance/

[3] J. Balaram, M. Aung, and M. P. Golombek, "The Ingenuity Helicopter on the Perseverance Rover," *Space Science Reviews*, vol. 217, 2021, art. no. 56. [Online]. Available: https://doi.org/10.1007/s11214-021-00815-w

[4] NASA Jet Propulsion Laboratory, "Ingenuity Mars Helicopter," consultado em 8 nov. 2026. [Online]. Available: https://www.jpl.nasa.gov/missions/ingenuity/

[5] M. P. Golombek *et al.*, "Overview of the Mars Pathfinder Mission and Assessment of Landing Site Predictions," *Science*, vol. 278, no. 5344, pp. 1743–1748, 1997. [Online]. Available: https://doi.org/10.1126/science.278.5344.1743

[6] NASA, "Mars Pathfinder," consultado em 8 nov. 2026. [Online]. Available: https://science.nasa.gov/mission/mars-pathfinder/

[7] D. Dorf and K. Hughes, "spaceros_gz_demos," consultado em 8 nov. 2026. [Online]. Available: https://github.com/david-dorf/spaceros_gz_demos

[8] Journey Man, "Mars Pathfinder Lander (HIGH Detail)," SketchUp 3D Warehouse, modelo 3D, consultado em 8 nov. 2026. [Online]. Available: https://3dwarehouse.sketchup.com/model/u369e1cb8-af6e-46dc-9d4d-57a919f5b2ca/Mars-Pathfinder-Lander-HIGH-Detail?hl=en

[9] S. Macenski, T. Foote, B. Gerkey, C. Lalancette, and W. Woodall, "Robot Operating System 2: Design, Architecture, and Uses in the Wild," *Science Robotics*, vol. 7, no. 66, 2022, art. no. eabm6074. [Online]. Available: https://doi.org/10.1126/scirobotics.abm6074

[10] Open Robotics, "ROS 2 Jazzy Jalisco Documentation," consultado em 8 nov. 2026. [Online]. Available: https://docs.ros.org/en/jazzy/

[11] N. Koenig and A. Howard, "Design and Use Paradigms for Gazebo, an Open-Source Multi-Robot Simulator," in *Proc. IEEE/RSJ Int. Conf. Intelligent Robots and Systems (IROS)*, 2004, pp. 2149–2154. [Online]. Available: https://doi.org/10.1109/IROS.2004.1389727

[12] Open Robotics, "Gazebo Harmonic Documentation," consultado em 8 nov. 2026. [Online]. Available: https://gazebosim.org/docs/harmonic/

[13] ——, "SDFormat Specification," consultado em 8 nov. 2026. [Online]. Available: https://sdformat.org/spec/

[14] ——, "ros_gz_bridge, ROS 2 Jazzy," consultado em 8 nov. 2026. [Online]. Available: https://docs.ros.org/en/jazzy/p/ros_gz_bridge/

[15] ——, "rclcpp, ROS 2 Jazzy," consultado em 8 nov. 2026. [Online]. Available: https://docs.ros.org/en/jazzy/p/rclcpp/

[16] PX4 Autopilot, "ROS 2 Offboard Control Example," consultado em 8 nov. 2026. [Online]. Available: https://docs.px4.io/main/en/ros2/offboard_control

[17] ——, "ROS 2 User Guide," consultado em 8 nov. 2026. [Online]. Available: https://docs.px4.io/main/en/ros2/user_guide

[18] eProsima, "eProsima Micro XRCE-DDS Agent," consultado em 8 nov. 2026. [Online]. Available: https://micro-xrce-dds.docs.eprosima.com/en/latest/agent.html

[19] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You Only Look Once: Unified, Real-Time Object Detection," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 779–788. [Online]. Available: https://doi.org/10.1109/CVPR.2016.91

[20] Open Robotics, "geometry_msgs/msg/PointStamped, ROS 2 Jazzy," consultado em 8 nov. 2026. [Online]. Available: https://docs.ros.org/en/jazzy/p/geometry_msgs/msg/PointStamped.html

[21] ——, "Gazebo Sim 8 API Reference: OdometryPublisher," consultado em 8 nov. 2026. [Online]. Available: https://gazebosim.org/api/sim/8/classgz_1_1sim_1_1systems_1_1OdometryPublisher.html

[22] ——, "nav_msgs/msg/Odometry, ROS 2 Jazzy," consultado em 8 nov. 2026. [Online]. Available: https://docs.ros.org/en/jazzy/p/nav_msgs/msg/Odometry.html

[23] ——, "geometry_msgs/msg/Twist, ROS 2 Jazzy," consultado em 8 nov. 2026. [Online]. Available: https://docs.ros.org/en/jazzy/p/geometry_msgs/msg/Twist.html

[24] ——, "sensor_msgs/msg/PointCloud2, ROS 2 Jazzy," consultado em 8 nov. 2026. [Online]. Available: https://docs.ros.org/en/jazzy/p/sensor_msgs/msg/PointCloud2.html

[25] ——, "Understanding Topics, ROS 2 Jazzy," consultado em 8 nov. 2026. [Online]. Available: https://docs.ros.org/en/jazzy/Tutorials/Beginner-CLI-Tools/Understanding-ROS2-Topics/Understanding-ROS2-Topics.html

[26] ANYbotics, "Grid Map Library," consultado em 8 nov. 2026. [Online]. Available: https://github.com/ANYbotics/grid_map

[27] P. E. Hart, N. J. Nilsson, and B. Raphael, "A Formal Basis for the Heuristic Determination of Minimum Cost Paths," *IEEE Trans. Syst. Sci. Cybern.*, vol. 4, no. 2, pp. 100–107, 1968. [Online]. Available: https://doi.org/10.1109/TSSC.1968.300136

[28] J. Carsten, A. Rankin, D. Ferguson, and A. Stentz, "Global Path Planning on Board the Mars Exploration Rovers," in *Proc. IEEE Aerospace Conf.*, 2007, pp. 1–11. [Online]. Available: https://doi.org/10.1109/AERO.2007.352683